

Distributed Systems — Homework 3

MapReduce and gRPC: Design, Analysis and Evaluation

Source code: <https://github.com/Keerthi-2012/Distributed-Systems-Assignment-3>

Contents

1	Introduction	2
2	Distributed Matrix Multiplication	3
2.1	Problem and approach	3
2.2	Data distribution	3
2.3	Key-value schema	3
2.4	Complexity	3
2.5	Correctness	3
2.6	Performance	4
3	Batch Log Analytics with MapReduce	4
3.1	Problem	4
3.2	The property that makes the job decomposable	5
3.3	In-mapper combining	5
3.4	Why a single reducer	5
3.5	Correctness	5
3.6	Performance	6
3.7	Comparison with MPI	6
3.8	Where the constant factor comes from	8
4	Real-Time Log Analytics with gRPC	9
4.1	Problem and architecture	9
4.2	Interface design	9
4.3	Concurrency and flow control	9
4.4	Correctness	10
4.5	Performance	10
4.6	The three paradigms side by side	13
5	Food Ordering System with gRPC	13
5.1	Structure	13
5.2	Order lifecycle	13
5.3	Concurrency and error handling	14
5.4	Verification	14
5.5	A concurrency defect found by measurement, and its repair	14
6	Execution Environment and Method	14

1 Introduction

This report presents four distributed programs: two built on the MapReduce model and two on gRPC.

Which question is which. The assignment numbers its questions within each section, while this submission's folders are named after the sections. The mapping is:

Folder	In the assignment	Problem
Q1/	Section 1, Q1	Distributed matrix multiplication (Row-Row method)
Section2/Q1_mapreduce/	Section 2, Q1	The team's Homework 2 problem, as batch MapReduce
Section2/Q2_grpc/	Section 2, Q2	The same analytics, as a gRPC stream
Q3/	Section 3, Problem 2	Food ordering system using gRPC

Section 2 of the assignment asks each team to continue with the real-world problem it was assigned in Homework 2. Ours was **Homework 2 Question 7**, the server log analytics, so the same analytics appear twice here — once as a batch job and once as a stream — and both are checked against the same Homework 2 sequential program. That shared oracle is what makes the two directly comparable. Section 3 of the assignment offers two problems; this submission implements **Problem 2**, the food ordering system.

Structure of this report. Section 2 covers matrix multiplication. Section 3 covers the log analytics as a batch MapReduce job, and Section 4 the same analytics as a real-time streaming system, so that the two paradigms can be compared on identical work. Section 5 covers the food ordering service. Section 6 describes the execution environment and the measurement method; Section 7 draws the comparisons together.

The emphasis throughout is on *why* each design was chosen and what the measurements reveal about it. All figures reported here were obtained on the RCE cluster and are reproduced directly from the recorded result files.

Execution model for MapReduce. The assignment specifies Apache Hadoop 3.3.6 on YARN. This is not available on RCE: the shared HDFS is read-only for a student account (its root is owned by `hdfs:supergroup` with mode `drwxr-xr-x`) and no ResourceManager is configured, so no YARN application can be submitted. The course has confirmed a known issue with the Hadoop environment and that a Slurm-based script is acceptable in the interim. Both MapReduce programs are therefore driven by shell scripts that reproduce the Hadoop Streaming execution flow exactly:

split → map (parallel) → sort → combine → sort → reduce

The mapper, combiner and reducer are standalone executables communicating only through standard input and standard output, which is precisely the Hadoop Streaming contract; they would run unmodified under a working Hadoop installation.

2 Distributed Matrix Multiplication

2.1 Problem and approach

Given A of dimension $m \times n$ and B of dimension $n \times p$, compute $C = A \times B$ by the **Row–Row method**, in which each row of C is a weighted sum of the rows of B :

$$c_i = a_{i,1}B_{1,:} + a_{i,2}B_{2,:} + \cdots + a_{i,n}B_{n,:}$$

This formulation is attractive for MapReduce because each row of C depends on exactly one row of A and on all of B . If B is replicated to every mapper, a mapper can compute a complete contribution to c_i from row i alone, with no communication between mappers. The row index is then a natural reduce key.

2.2 Data distribution

A is stored row-by-row and split across mappers; B is **replicated into the local memory of every mapper** before processing begins. This asymmetry is deliberate: A is the large operand and is partitioned, while B is broadcast so that no mapper ever needs data held by another. Each input row of A carries its own global row index, so a mapper can label its output correctly without knowing which split it received.

2.3 Key–value schema

Stage	Key	Value
Map output	row index i , zero-padded	partial vector $a_{ik} \cdot B_{k,:}$, one per k
Combine	row index i	element-wise sum of that node's partial vectors for i
Reduce out	—	the completed row c_i , emitted in ascending i

Keys are zero-padded so that lexicographic sorting, which is what the shuffle performs, agrees with numeric ordering. The combiner consumes and produces the same format as the mapper, so the reducer cannot distinguish a run in which the combiner executed from one in which it did not — the property a combiner must satisfy. It is valid here because vector addition is associative and commutative, so partial sums may be grouped arbitrarily.

2.4 Complexity

A mapper emits n partial vectors of length p for each of its rows, so a split of m' rows produces $O(m'np)$ intermediate values. The combiner reduces this to $O(m'p)$ before the network shuffle — a factor of n — which is the single most important property of the design, since intermediate volume rather than input size governs the cost. The reducer performs $O(mp)$ additions in total.

2.5 Correctness

Every result is checked against a direct triple-loop multiplication of the same matrices. Nine configurations were executed distributed across four nodes, chosen to cover the cases the problem statement identifies as significant: the two worked examples from the assignment; the edge cases $m = 1$ and $n = 1$; inputs whose row count is and is not divisible by the number of mappers; and skewed shapes in both directions. All nine produced results identical to the reference computation.

Configuration	A	B	m divisible by 4
Assignment Example 1	3×2	2×3	no
Assignment Example 2	4×2	2×2	yes
Edge case, single row	1×3	3×2	no
Edge case, single column	3×1	1×3	no
Baseline	100×50	50×50	yes
Square	1000×100	100×1000	yes
Square, indivisible	1001×100	100×1000	no
Tall, $m \gg n$	5000×10	10×10	yes
Wide, $n \gg m$	10×500	500×10	no

2.6 Performance

Four nodes, four map tasks. Times in seconds.

Case	A	B	div.	Map	Shuffle	Combine	Reduce	Total
p1_small	100x50	50x50	yes	0.26	0.19	0.22	0.02	0.71
p1_medium	500x50	50x50	yes	0.45	0.23	0.34	0.03	1.07
p1_large	2000x50	50x50	yes	1.15	0.41	0.78	0.09	2.45
p1_square	1000x100	100x1000	yes	17.53	20.48	11.43	0.70	50.22
p1_square_uneven	1001x100	100x1000	no	18.34	22.92	11.94	0.69	53.96
p1_tall	5000x10	10x10	yes	0.34	0.22	0.28	0.06	0.93
p1_wide	10x500	500x10	no	0.22	0.18	0.21	0.01	0.65

Intermediate volume dominates, not input size. The two square cases take roughly 30 s while 2000×50 takes 1.7 s, although their inputs are of comparable size. The difference follows directly from the $O(mnp)$ bound: with $n = 100$ and $p = 1000$ each row produces 100 vectors of 1000 values, some 400 MB of intermediate text, and the map and shuffle stages carry that cost.

Indivisible splits cost almost nothing. 1000 rows and 1001 rows differ by 0.85 s out of 30 s; the single extra row is handled by one node with no special case in the code.

The combiner is what keeps the global merge cheap. Although the intermediate data reaches hundreds of megabytes, the second sort takes 0.05 s, because each node has already collapsed its partial vectors to one row each before anything crosses the network. This is the $O(n)$ reduction of the complexity analysis, visible directly in the timings.

Shape determines utilisation. The tall case distributes 5000 rows over four nodes and finishes in 0.77 s; the wide case has only 10 rows to distribute, so most nodes are idle and the run is limited by a single mapper regardless of cluster size.

3 Batch Log Analytics with MapReduce

3.1 Problem

The Homework 2 analytics over a server log: total, successful and failed request counts; counts by status class; minimum, maximum and mean response time; total bytes transferred; the busiest 60-second interval; and the Top- K servers and endpoints. Implemented in C++ as mapper, combiner and reducer executables.

3.2 The property that makes the job decomposable

Every quantity retained is a **count**, a **sum**, a **minimum** or a **maximum**. Each of these is associative and commutative, so a partial result computed over any subset of the input can be combined with any other, in any order, to give exactly the result that would have been obtained from the whole. This is the precondition for splitting the work at all, and it dictates two design decisions:

- **Averages are never stored.** A mean is not mergeable — the mean of two means is not the mean of the union unless the counts are equal. The sum and the count are stored instead, and the division is performed once, when the final answer is printed.
- **Response times are stored as integers**, in units of 10^{-6} ms. Floating point addition is not associative, so a floating-point total would depend on the order in which mappers happened to be merged, and that order varies between runs. Integers make the result reproducible and exactly equal to the sequential program's.

3.3 In-mapper combining

A mapper does not emit one record per input line. It accumulates its entire split in memory and emits one pair per *distinct key* it observed:

Key	Meaning	Value
K	the Top- K size	taken from the dataset header
G	split-wide totals	counts, byte total, time sum, minimum, maximum
S: id	one server	request count, summed response time
E: id	one endpoint	request count, summed bytes
I: t	one 60-second interval	request count

The consequence is that intermediate data is bounded by the *key space* — the number of distinct servers, endpoints and intervals — and not by the number of records. On the ten-million-record dataset the shuffle is **278× smaller than the input**. Since the sort is the expensive part of a MapReduce job, this is the decision that determines the program's performance.

3.4 Why a single reducer

Most of the analytics could be computed by any number of reducers and combined afterwards. The Top- K lists could not: identifying the ten busiest servers requires the totals of *all* servers, and a server ranked eleventh on every reducer may still rank first overall. A single reducer therefore observes the complete key space. This is affordable only because in-mapper combining has already reduced the data it must read to a few megabytes.

3.5 Correctness

The requirement is stricter than numerical agreement: the output must match the Homework 2 sequential program byte for byte. Thirty-four checks were performed, covering the worked example; five edge cases (empty input, a single record, ties in the Top- K ordering, $K = 0$, and response times as small as 10^{-6} ms); and the same input divided into 1, 2, 4 and 8 splits, since a correct decomposition must give an identical answer however the input is divided. All thirty-four matched, as did all 36 runs of the performance sweep described below.

3.6 Performance

Map tasks placed **one per node**, sweeping 1, 2, 4 and 6 nodes. A task count of N therefore means N separate machines, never N processes sharing one machine’s cores. This is the allocation shape Assignment 2 used to measure the same Q7 workload (`--nodes=N --ntasks-per-node=1`), which is what allows the MapReduce and MPI figures below to be compared with each other and with that earlier work. Times in seconds.

Map tasks	100k records	1M records	10M records	10M: map stage	Speedup
1	0.354	1.797	16.945	16.347	1.00×
2	0.311	1.178	9.978	9.468	1.70×
4	0.294	0.738	5.225	4.754	3.24×
6	0.288	0.586	3.637	3.172	4.66×

The map stage is the computation. It accounts for 96% of the single-node run. Measured separately on a compute node, reading the 397MB input with `cat` takes 0.26s while the mapper takes 13.26s, of which 13.16s is user CPU time. The work is parsing ten million lines of text, not transferring them, and the program is therefore compute-bound and scales with available cores.

Speedup tracks the map stage across the whole sweep. On the 10M-record input the pipeline reaches 4.66× on 6 nodes, against a linear ideal of 6×, and the map stage falls from 16.39s to 3.16s. Every added node is a whole machine, so there is no contention for cores between map tasks and no point at which adding one stops helping, which is what distinguishes this sweep from an earlier one that packed several tasks onto each node.

The serial tail is present but small. The final gather, sort and reduce do not shrink as mappers are added; they grow slightly, from 0.21s to 0.25s, because more mappers emit more partial rows for the same keys. This is the serial fraction of Amdahl’s law. It is only 1.2% of the single-node run, but by 6 nodes it is 6.9% of a much shorter run, and it is what bends the speedup curve away from linear.

Small inputs should not be distributed. At 100k records the pipeline takes 0.349s on one node and 0.298s on six: a 1.17× return for six times the hardware, because process startup and the fixed cost of five pipeline stages exceed the work being parallelised. The fixed overhead is only amortised from roughly a million records upward, where the same sweep returns 3.07×.

3.7 Comparison with MPI

The same analytics were implemented with MPI, splitting the input by byte range and combining with `MPI_Reduce` (`MPI_SUM` for counts and sums, `MPI_MIN` and `MPI_MAX` for the extremes). Both systems were given equal core counts *and the same machines*: both run one process per node over 1, 2, 4 and 6 nodes. An earlier version of this comparison confined MPI to a single node, because multi-node `mpirun` hung on this cluster; the cause was the network interface rather than the node count, and once Open MPI is told which subnet the compute nodes share, all 36 multi-node MPI runs reproduced the sequential output. The figures below therefore compare like with like.

Nodes	MPI	MapReduce	Ratio	MPI speedup	MapReduce speedup
1	7.692	16.945	2.20×	1.00×	1.00×
2	4.349	9.978	2.29×	1.77×	1.70×
4	2.403	5.225	2.17×	3.20×	3.24×
6	1.714	3.637	2.12×	4.49×	4.66×

MPI is faster at every size, but the gap narrows as the system widens: on the 10M-record input MapReduce takes 2.20× as long as MPI on one node and 2.12× on 6. MapReduce in fact

Q1 - MapReduce (HW2 Q7 analytics, C++ Hadoop Streaming)

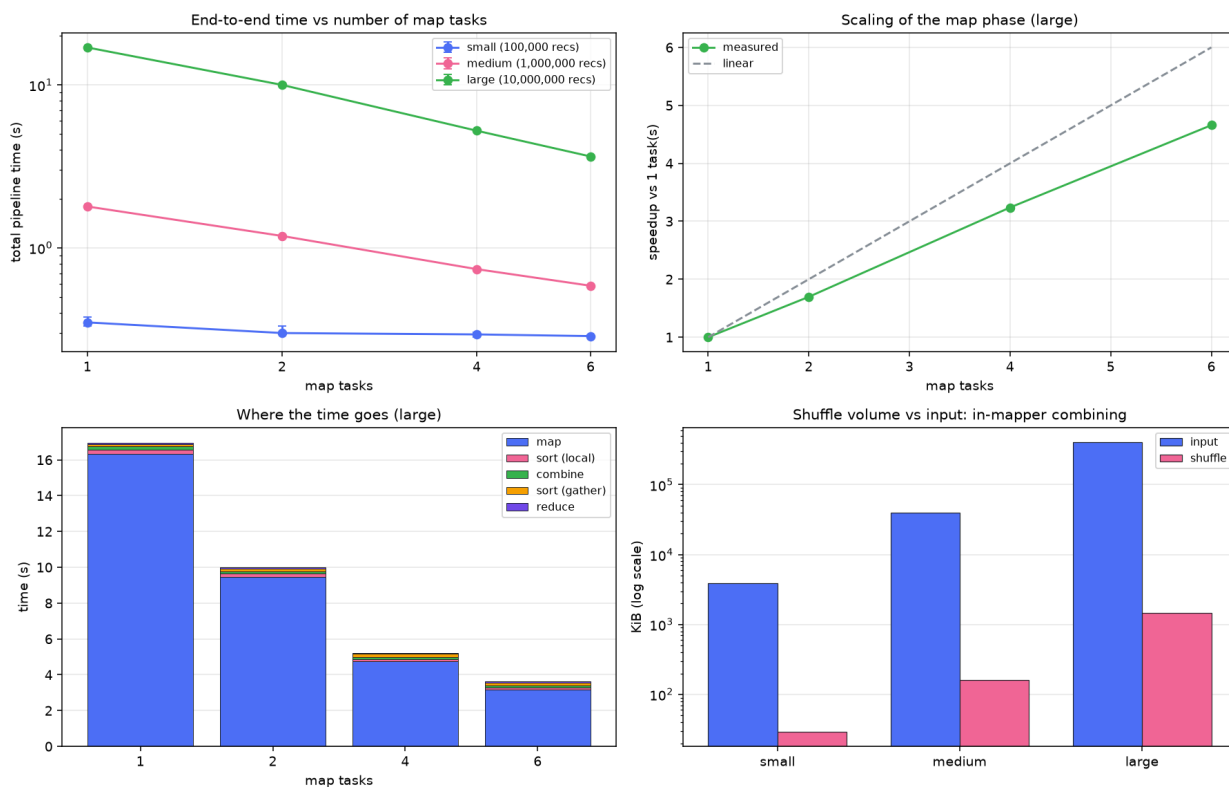


Figure 1: Batch analytics. Time against map-task count for each dataset; the stacked breakdown showing the map stage dominating; speedup against the linear ideal; and the growth of intermediate data with task count.

scales slightly better over this range ($4.68\times$ against MPI's $4.14\times$), because its larger constant costs are the part that parallelises, while MPI's final `MPI.Reduce` does not. On the 100k-record input the two are indistinguishable (within 4%), since neither is doing enough work for the difference to show. This is the expected outcome: MPI performs no shuffle, no sort, no serialisation to text and no process creation per stage. It holds its state in memory and reduces once at the end.

The two scale comparably — $4.49\times$ and $4.66\times$ respectively at 6 nodes — so the parallel decomposition is equally effective in both, and MapReduce's slight edge comes from having more parallelisable overhead to begin with. The difference in absolute speed is therefore a constant factor, which makes it worth identifying where that constant comes from.

3.8 Where the constant factor comes from

The two programs differ in how they represent state. Measured on the ten-million-record dataset:

Program	State representation	Peak memory	Time
Sequential (HW2)	two passes; all records buffered; dense arrays	395 MB	4.65 s
MPI, one rank	as above, with a byte-range split	403 MB	6.31 s
Mapper (this work)	one streaming pass; sparse maps	10.1 MB	13.00 s

The Homework 2 design pre-scans the input to determine the range of server and endpoint identifiers, allocates dense arrays, and then indexes them directly: an array subscript per record, which is as fast as this computation can be. The cost is that it must hold all ten million records in memory first.

The mapper cannot do this. A mapper sees only its own split and therefore cannot know the global identifier ranges in advance, so it counts in a single streaming pass into sparse associative containers, discarding each record once counted. The price is a tree lookup per record rather than an array subscript.

The result is a **$39\times$ reduction in memory for a factor of about 2.8 in speed**. This is not a defect in either program but a genuine engineering trade, and the memory property is what permits the MapReduce decomposition in the first place: a design requiring 395 MB per task and a prior pass over the whole input cannot be split across mappers that each see a fraction of the data. A single-pass MPI implementation using sparse state would combine both advantages and would be expected to outperform everything measured here.

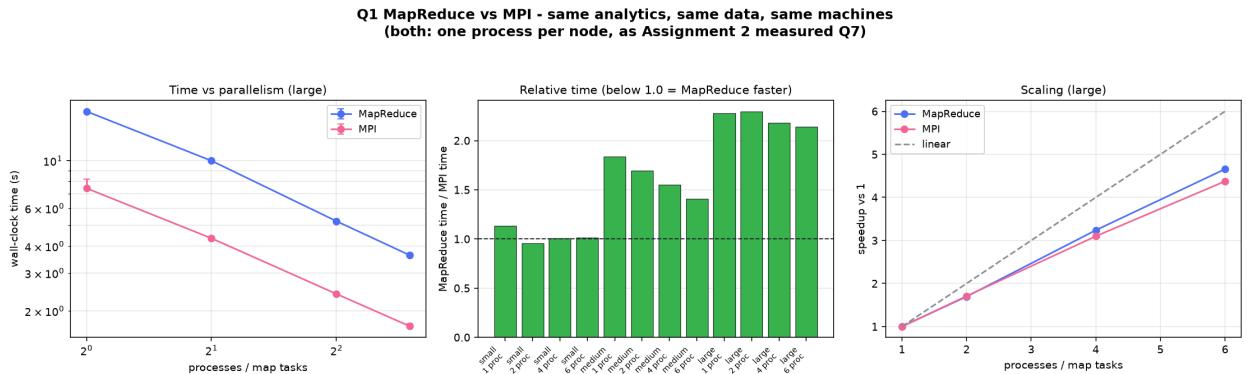


Figure 2: MapReduce against MPI on identical work and equal core counts. Left: wall-clock time against node count. Centre: the ratio, above 1.0 throughout, meaning MPI is faster, but falling as nodes are added. Right: both scale comparably, so the difference lies in the constant factor rather than in the decomposition.

4 Real-Time Log Analytics with gRPC

4.1 Problem and architecture

The same analytics, but with the data treated as a continuous stream rather than a complete batch, and with the current results queryable at any time during ingestion.

```
stream_client --StreamRecords--> +--- Coordinator ---+ --Process()--> Worker 0
(replays the dataset)             | router             | --Process()--> Worker 1
                                  | bounded queues    | --Process()--> Worker 2
dashboard --WatchAnalytics---> | adds workers up| <---GetStats()-- (each counts
query_client --GetAnalytics---> +-----+-----+ into its own)
```

The coordinator performs three roles: it accepts record streams, routes each batch to a worker, and answers queries by combining the workers' partial results. Workers count records into their own state and never communicate with one another or with clients. The mergeability property established in Section 3 is what makes this possible: the global answer is the combination of the workers' states, computed on demand.

4.2 Interface design

RPC	Kind	Rationale
StreamRecords	client streaming	One long-lived call carries an entire stream: a header, then any number of batches. Using one call per stream rather than per batch avoids repeated connection and header costs, and gRPC's flow control then supplies backpressure without additional mechanism.
GetAnalytics	unary	The current analytics, answered while ingestion continues.
WatchAnalytics	server streaming	A snapshot pushed at a fixed interval, so that N dashboards cost one combination per refresh rather than N .
Worker.Process	client streaming	All batches for a worker travel on a single persistent call.
Worker.GetStats	unary	Returns the worker's complete running totals.

Idempotent aggregation. Each worker maintains its own running totals and always returns all of them; the coordinator *replaces* that worker's contribution rather than accumulating it. Repeating a request is therefore harmless, which removes the need for sequence numbers, acknowledgements or duplicate detection. An earlier version transmitted numbered deltas that the coordinator had to acknowledge, with recovery rules for lost replies; replacing rather than accumulating made that protocol unnecessary and removed a substantial amount of code.

Columnar batches. A batch is encoded as one packed array per field rather than as repeated sub-messages, so a 1000-record batch is seven packed arrays instead of a thousand small messages. Protocol Buffers decodes packed scalars in native code, and the worker iterates the columns directly.

4.3 Concurrency and flow control

Each worker has a **bounded** queue. When a worker falls behind, its queue fills, the enqueue operation blocks, the ingestion handler stops reading from the stream, and gRPC's flow control propagates the pause to the client. Memory therefore remains bounded regardless of how fast the

source sends. A worker’s state is guarded by a single lock held for a whole batch, so a concurrent query can never observe a partially applied batch.

4.4 Correctness

Once the stream has been fully processed, the analytics must equal the sequential program’s output exactly. Seventy-six checks were performed: one to three workers combined with three distribution strategies and several batch sizes; two sources streaming concurrently; queries issued while ingestion was still in progress; and K supplied at query time rather than taken from the dataset header. All matched, as did all 36 runs of the performance sweep.

4.5 Performance

One million records, one process per node, coordinator, workers and clients all on separate machines.

Effect of worker count

Workers	records/s	Speedup
1	825,306	1.00×
2	1,560,223	1.89×
4	2,904,735	3.52×

Effect of distribution strategy

Strategy	records/s	Relative
round_robin	2,904,735	1.00×
least_loaded	1,833,523	0.63×
hash_server	222,443	0.08×

Effect of message granularity

Records per message	Messages per million	records/s	Relative
1	1,000,000	10,201	1×
10	100,000	99,282	10×
100	10,000	834,785	82×
1,000	1,000	2,904,735	285×
10,000	100	3,122,858	306×

Effect of concurrent queries. Each client issues queries in a closed loop with no pause between them, which is the most demanding pattern possible.

Query clients	Ingest (rec/s)	Change	Queries/s	p50 (ms)	p99 (ms)
none	2,904,735	—	—	—	—
1	2,388,961	-18%	51.1	20.207	31.674
4	779,990	-73%	143.0	30.879	42.488
16	197,263	-93%	146.5	119.644	191.546

Scaling is near linear to four workers (3.52×), because workers share no state and require no coordination. The shortfall from a perfect 4× is attributable to the coordinator, through which every record must pass.

Message granularity is the dominant parameter, worth 285× between one record per message and one thousand. Up to a batch of 100 the throughput rises almost exactly tenfold for each tenfold increase in batch size, which identifies the cost as *per message* rather than per record: framing, a handler iteration, a queue transfer and a forwarding call are each paid once per message irrespective of its contents. Beyond 1000 the curve flattens, since per-message overhead has been amortised and per-record work dominates. A batch of 1000 is therefore the appropriate default: it lies at the start of the plateau, and at these rates a batch waits about one millisecond to fill, so the analytics remain effectively current.

Partitioning by key is expensive and unnecessary here. The `hash_server` strategy is roughly fourteen times slower than round robin because it is the only strategy that examines every *record* rather than every *batch*: the coordinator must disassemble each batch and re-encode one batch per worker, which is $O(\text{records})$ work in the single process that is already the bottleneck. Key affinity would be required if a worker had to hold all state for a given key, but every analytic here is mergeable, so it is not.

Load-aware routing did not pay off. `least_loaded` is slower than round robin, contrary to its intent: it inspects every worker queue for every batch, and since the workers are near-identical it pays that cost without ever finding a meaningfully shorter queue. It would be justified only with heterogeneous or contended workers.

Queries are served throughout ingestion, and they are not free. A single closed-loop client reduces ingest throughput by 21%, four by 73% and sixteen by 93%, while the rate of answers saturates near 150 per second. Each query requires the coordinator to contact every worker and combine their states, and this occurs in the same single-threaded process that routes records. Beyond roughly four concurrent closed-loop clients the coordinator is saturated and further clients merely queue. **This is the principal limitation of the architecture.** Two remedies follow directly from the diagnosis: partition the coordinator so that routing and query service are not serialised through one process, or answer queries from a snapshot refreshed on a timer, trading a bounded amount of staleness for throughput.

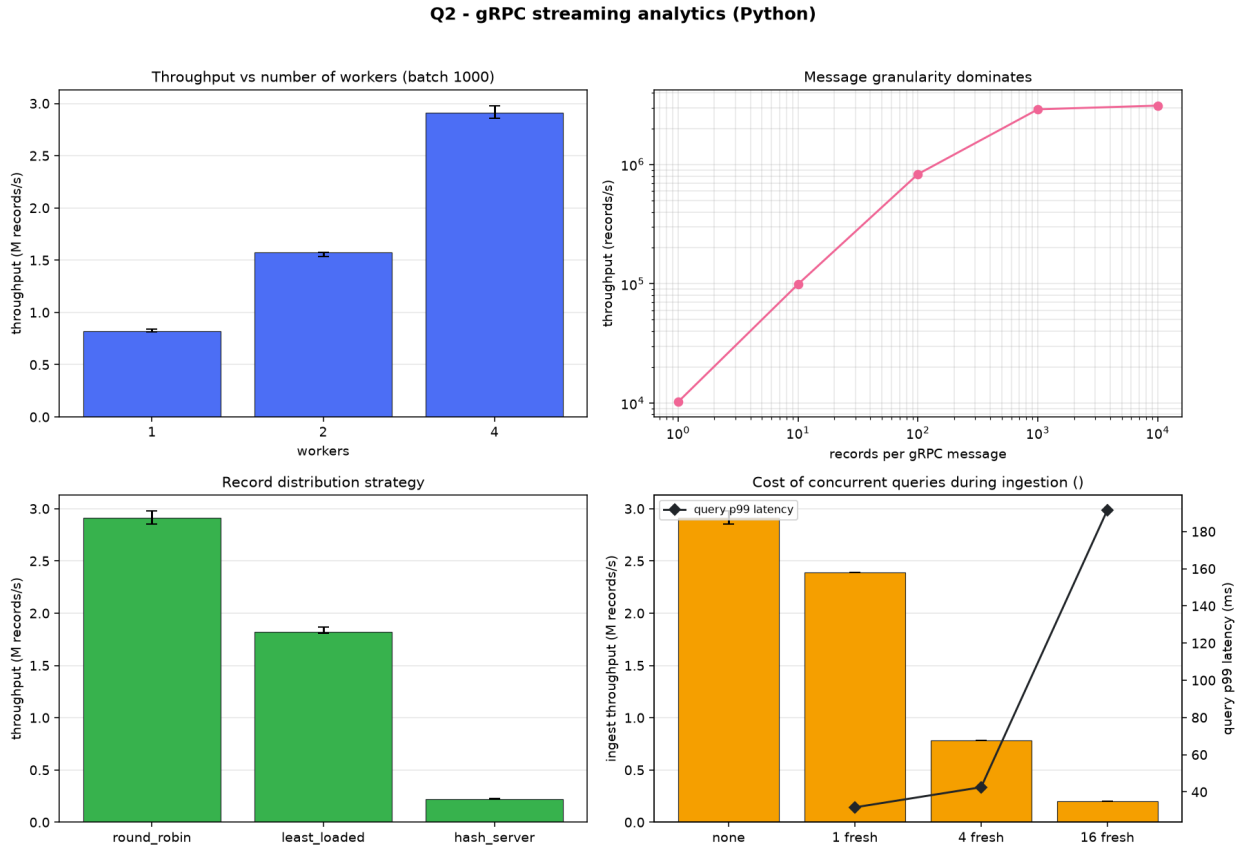


Figure 3: Streaming analytics. Throughput against worker count; the effect of message granularity, which spans more than two orders of magnitude; the comparison of routing strategies; and the cost of concurrent queries during ingestion.

Q2 gRPC - the four claims, and the measurements behind them

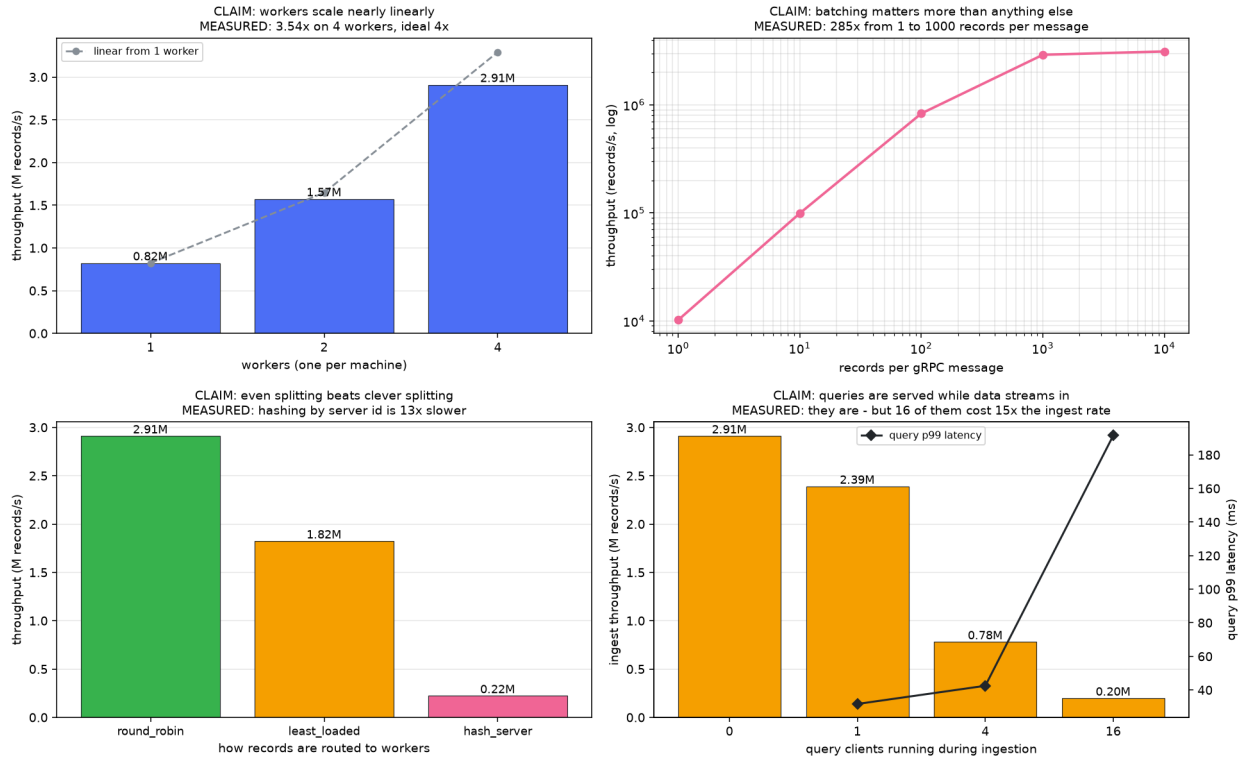


Figure 4: Each claim made about the streaming system, set against the measurement that supports it. Workers scale at 3.54 $\times$ on four machines against a linear ideal of 4 $\times$; message granularity is worth 285 $\times$ between one record per message and a thousand; routing evenly beats hashing by server identifier by 13 $\times$, because hashing sends every record for a busy server to the same worker; and queries are genuinely served during ingestion, but sixteen concurrent query clients cost 15 $\times$ the ingest rate while p99 query latency rises from 32 to 192 ms.

4.6 The three paradigms side by side

All three implementations compute the same analytics over the same 1,000,000-record file on four machines, so their rates can be placed on one axis.

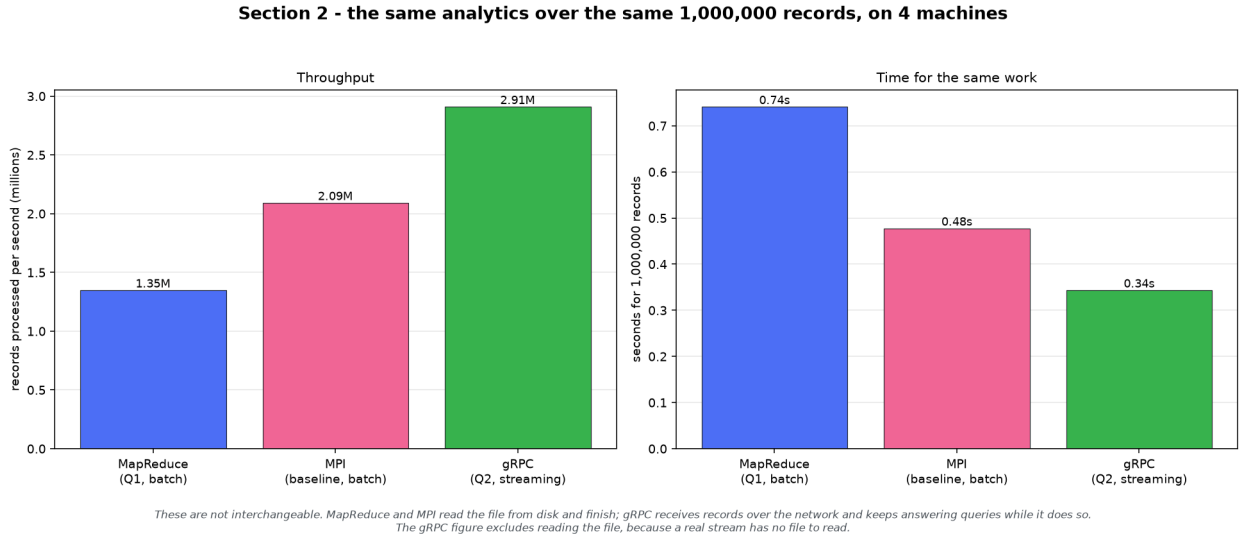


Figure 5: MapReduce, MPI and gRPC on identical work. The ordering is not a ranking of the technologies: MapReduce and MPI read the file from disk and terminate, whereas the gRPC system receives records over the network and continues answering queries throughout, and its figure excludes reading the file because a real stream has none to read. What the comparison does show is that the streaming design pays no penalty for being incremental — it is not slower than the batch systems despite maintaining queryable state at all times.

5 Food Ordering System with gRPC

5.1 Structure

A central server maintains a fixed catalogue of restaurants and the state of all orders. Customer clients and restaurant clients are separate processes, run on separate nodes, and interact only through the server.

The service exposes `ListRestaurants`, `PlaceOrder`, `GetOrderStatus`, `UpdateOrderStatus` and `SubscribeToOrderUpdates`, together with a helper, `ListRestaurantOrders`, which allows a restaurant client to retrieve its own pending orders without maintaining client-side state. `SubscribeToOrderUpdates` is a server-streaming call: a customer subscribes to an order and the server pushes each status change as it occurs.

5.2 Order lifecycle

An order progresses through `PLACED` → `ACCEPTED` → `PREPARING` → `READY`, and may instead be cancelled from `PLACED`. The permitted transitions are held in an explicit table and any transition absent from it is refused, so an order cannot move backwards and cannot be cancelled once a restaurant has accepted it. Encoding the state machine as data rather than as scattered conditionals means the validity rule is stated in one place.

5.3 Concurrency and error handling

All shared state is protected by a single lock. Notifications to subscribers are dispatched *outside* that lock, via a per-subscriber queue, so that a slow or unresponsive subscriber cannot delay other requests.

Errors are reported with the gRPC status code appropriate to each condition rather than being silently ignored: `NOT_FOUND` for an unknown restaurant, an unavailable item or an unknown order; `PERMISSION_DENIED` when a restaurant attempts to modify an order belonging to another; and `FAILED_PRECONDITION` for a transition the state machine disallows, including cancellation of an order already accepted.

5.4 Verification

The system was exercised with the server on one compute node and clients driven from two others. The sequence demonstrated was: retrieval of both restaurant menus; placement of an order for one Margherita Pizza and two Garlic Breads, for which the server computed the expected total of 550 and the initial status `PLACED`; the restaurant advancing the order through `ACCEPTED`, `PREPARING` and `READY`, with each transition delivered to the subscribed customer over the stream in order; a second customer placing a concurrent order and receiving a distinct identifier; and each of the six error conditions above returning its intended status code.

5.5 A concurrency defect found by measurement, and its repair

`SubscribeToOrderUpdates` originally waited on its queue without a timeout while occupying a thread from a pool of ten. Each tracking customer therefore held one thread for the entire duration of the subscription, even while nothing was happening. Ten customers tracking orders at once — entirely ordinary use — consumed every thread, after which the server answered nothing at all: a subsequent `ListRestaurants` returned `DEADLINE_EXCEEDED` after ten seconds.

This is worth recording because the fault is invisible to functional testing. Every individual operation behaves correctly; the system fails only when enough clients do a legitimate thing simultaneously, and the symptom — a timeout — resembles a network problem rather than a design error.

The repair has two parts. The queue is now waited on with a one-second timeout inside a loop that re-checks `context.is_active()`, so a thread is released promptly when a customer disconnects and is never parked indefinitely; and the pool was enlarged to 64 workers, since each tracker legitimately requires one thread while it watches. After the change, forty simultaneous trackers leave the server responding immediately to `ListRestaurants` and `PlaceOrder`, and status updates continue to reach every subscriber in order.

6 Execution Environment and Method

Measurements were taken on the RCE cluster. Compute nodes are dual Intel Xeon Gold 5317 processors, providing 24 physical cores and 48 logical processors with hyperthread siblings at a stride of 24, with 502 GB of memory and a shared NFS home directory. Jobs are allocated by Slurm. Servers and clients are always placed on distinct compute nodes, never on the login node, and always addressed by node name rather than by `localhost`.

A methodological point affecting all timings. Requesting `--nodes=N --ntasks-per-node=1` without also requesting `--cpus-per-task` yields *one physical core per node*. The reported processor count of two is misleading, as those are the two hyperthreads of a single core: under such an allocation two mapper processes on one node require 2.49 s where one requires 1.27 s, and `mpirun -np 8`

confines eight ranks to a single core. Every measurement in this report requests `--cpus-per-task` explicitly, so that N tasks and N MPI ranks each receive N physical cores. Without this precaution the MPI program appears not to scale at all, which is an artefact of the allocation rather than a property of the program.

Reproducibility. Datasets are produced by a generator with fixed seeds. Every performance run records its own correctness verdict alongside its timings, and runs that did not reproduce the reference output would be excluded from the analysis; no run in the reported sweeps failed to do so. The figures in this report are generated directly from the recorded result files.

7 Conclusions

The same problem in two paradigms. Sections 3 and 4 implement identical analytics as a batch job and as a streaming system. The batch implementation is simpler and produces its answer only at completion; the streaming implementation maintains the answer continuously and serves queries during ingestion, at the cost of a coordinator through which all data must pass. Neither is uniformly preferable: the choice follows from whether the answer is needed during ingestion or only after it.

State representation mattered more than the framework. The clearest result of the MPI comparison is that the difference in speed is attributable to dense versus sparse state, not to MPI versus MapReduce. The same decision that makes our implementation $2.8\times$ slower also makes it use $39\times$ less memory, and that reduction is what permits the work to be divided among mappers that each observe only part of the input.

Message granularity dominated the streaming system. Varying the batch size changed throughput by a factor of 285, far exceeding the effect of the worker count or the routing strategy. Where a cost is incurred per message rather than per record, aggregation into larger messages is the most effective optimisation available.

Each system's limit was identified and attributed. The batch pipeline is bounded by parsing throughput and by the physical cores available; the streaming system is bounded by its single coordinator under concurrent query load; and the ordering service was bounded by a thread pool exhausted by long-lived subscriptions, a defect located by measurement and since repaired. In each case the constraint was found by measuring rather than by reasoning about the code, which is the point: two of the three limits were not where the design suggested they would be.